



01 설정과 시작

커밋에 기록할 이름·이메일 설정

```
git config --global user.name "Name"
git config --global user.email "me@host.com"
```

--global: 내 계정의 모든 저장소에 적용. 생각하면 현재 저장소에만 적용.

현재 폴더를 Git 저장소로 초기화

```
git init
```

원격 저장소를 로컬로 복제

```
git clone <url>
```

02 변경을 기록하기

파일의 변경 내용을 스테이징

```
git add <file>
```

현재 폴더 아래의 변경을 모두 스테이징

```
git add .
```

새 파일·수정·삭제 모두 포함.

스테이징된 내용을 커밋으로 저장

```
git commit -m "message"
```

추적 중인 파일의 수정·삭제를 커밋

```
git commit -am "message"
```

-a: 자동 스테이징. 새 파일은 포함하지 않음.

이동·이름 변경 / 파일 삭제 후 스테이징

```
git mv <old> <new>
git rm <file>
```

파일은 유지하고 Git 추적만 제거

```
git rm --cached <file>
```

기본 용어

main: 관례적으로 쓰는 기본 브랜치 이름
origin: clone 시 기본 원격 이름
HEAD: 현재 체크아웃한 커밋을 가리킴
HEAD~1: 첫 번째 부모 / HEAD~3: 3대 위
.gitignore: 새 파일의 추적 제외 규칙

03 브랜치와 태그

로컬 / 원격 / 전체 브랜치 목록

```
git branch
git branch -r
git branch -a
```

브랜치 생성 / 브랜치 전환

```
git branch <branch>
git switch <branch>
```

브랜치를 새로 만들고 바로 전환

```
git switch -c <branch>
```

현재 브랜치 이름 변경

```
git branch -m <new-name>
```

로컬 브랜치 삭제 / 강제 삭제

```
git branch -d <branch>
git branch -D <branch>
```

-d: 병합 여부 검사. -D: 검사 없이 강제 삭제.

현재 커밋에 버전 이름(태그) 부여

```
git tag <tag-name>
```

04 병합 (merge)

feature의 변경을 main에 병합

```
git switch main
git merge feature
```

병합 결과는 현재 브랜치(main)에 반영.

가능해도 fast-forward 대신 병합 커밋 생성

```
git merge --no-ff feature
```

변경을 합쳐 한 커밋으로 기록

```
git merge --squash feature
git commit -m "message"
```

--squash 자체는 커밋을 생성하지 않음.

충돌 해결 후 병합 마무리

```
git add <file>
git merge --continue
```

진행 중인 병합을 중단

```
git merge --abort
```

05 리베이스 (rebase)

커밋을 새 기준 위에 다시 적용. 재작성된 커밋은 ID가 바뀜.

feature 커밋을 main의 최신 커밋 위로 이동

```
git switch feature
git rebase main
```

최근 3개 커밋 순서·메시지·합치기 편집

```
git rebase -i HEAD~3
```

충돌 해결 후 계속 / 리베이스 중단

```
git add <file>
git rebase --continue
git rebase --abort
```

06 되돌리기

스테이징 취소 (파일 수정 내용은 유지)

```
git restore --staged <file>
```

스테이징하지 않은 변경을 index 버전으로 복원

```
git restore <file>
```

해당 파일의 스테이징하지 않은 변경을 버림.

과거 커밋을 확인 (브랜치에서 분리)

```
git switch --detach <commit>
```

읽기 전용이 아님. 작업을 유지하려면 새 브랜치 생성.

커밋의 변경을 취소하는 새 커밋 생성

```
git revert <commit>
```

기존 커밋 이력을 유지. 공유한 변경의 취소에 적합.

직전 커밋 취소 + 스테이징 상태 유지

```
git reset --soft HEAD~1
```

직전 커밋 취소 + 스테이징 해제

```
git reset --mixed HEAD~1
```

파일의 수정 내용은 유지. --mixed가 기본값.

직전 커밋 취소 + 파일도 이전 상태로 복원

```
git reset --hard HEAD~1
```

추적 파일의 변경을 잃음. 일부 미추적 파일도 덮어쓸 수 있음.

merge / rebase / revert 구분

merge: 브랜치의 변경을 합침
rebase: 커밋을 새 기준 위에 재적용
revert: 변경을 상쇄하는 커밋 추가
공유한 커밋의 reset/rebase는 협업자와 조율.

07 상태와 이력 확인

수정·새 파일·스테이징 상태 확인

```
git status
```

커밋 이력을 간단히 / 그래프로 확인

```
git log --oneline
git log --oneline --graph --all
```

스테이징 전 / 스테이징된 변경 비교

```
git diff
git diff --cached
```

두 커밋 사이 변경 비교 / 커밋 상세 확인

```
git diff <commit1> <commit2>
git show <commit>
```

08 임시 보관 (stash)

추적 파일의 변경을 임시 보관

```
git stash push -m "message"
```

작업 디렉터리·스테이징 변경을 보관하고 정리.

미추적 파일 포함 / 무시된 파일까지 포함

```
git stash push -u
git stash push -a
```

변경의 일부만 골라 임시 보관

```
git stash push -p
```

보관 목록 / 선택한 보관 내용의 diff 확인

```
git stash list
git stash show -p "stash@{1}"
```

stash@{0}: 가장 최근. {1}: 그다음 항목.

적용하되 보관 항목 유지 / 성공 시 삭제

```
git stash apply
git stash pop
```

충돌이 나면 pop도 해당 항목을 유지.

특정 보관 항목 적용 / 삭제

```
git stash apply "stash@{1}"
git stash drop "stash@{1}"
```

모든 보관 항목 삭제

```
git stash clear
```

보관했던 변경을 잃을 수 있으므로 내용 확인.

09 원격 저장소 연결

origin 이름으로 원격 저장소 연결

```
git remote add origin <url>
```

원격 이름과 주소 확인

```
git remote -v
```

원격 이름 변경 / 연결 제거

```
git remote rename <old> <new>
git remote remove <remote>
```

remove는 로컬의 연결 설정을 제거.

10 동기화 (fetch / pull / push)

원격 이력 가져오기 (현재 브랜치 병합 없음)

```
git fetch origin
```

원격의 특정 브랜치 이력 가져오기

```
git fetch origin <branch>
```

원격 변경을 가져와 현재 브랜치에 병합

```
git pull --no-rebase origin main
```

main을 체크아웃한 상태에서 사용하는 예시.

원격 변경 위로 내 로컬 커밋을 재적용

```
git pull --rebase origin main
```

main을 체크아웃한 상태에서 사용하는 예시.

현재 브랜치 첫 push + 추적 관계 설정

```
git push -u origin <branch>
```

-u: 이후 pull-push의 기본 원격 브랜치를 설정.

추적 관계가 설정된 현재 브랜치 업로드

```
git push
```

특정 브랜치 / 태그 업로드

```
git push origin <branch>
git push origin <tag-name>
```

기존 checkout 명령어 대응

git checkout <branch> → git switch
git checkout -b <branch> → switch -c
파일 복원은 git restore로 구분.
Git은 로컬에서도 사용 가능.
GitHub는 원격 저장소를 제공하는 서비스.